



Banco de Dados I

SQL Continuação

INSTRUÇÕES DML – INSERT

Já demonstramos como criar uma tabela no banco de dados, agora mostraremos as maneiras básicas de se inserir dados nessa tabela. O comando usado para inserir dados é o INSERT. A síntese básica do comando INSERT é a seguinte:

INSERT INTO <nome da tabela> (campo1, campo2) VALUES (valor1, valor2);

Cada valor é inserido no campo que corresponde à posição do valor na lista: valor1 é inserido no campo1, valor2 no campo2 e assim por diante.

OBS: Os valores devem ser separados com uma vírgula e se o tipo do campo for texto deve está entre aspas duplas ou simples.

INSTRUÇÕES DML – INSERT

Vamos usar como exemplo a tabela “cliente” que criamos no banco de dados “venda”, se você não criou essa tabela volte para página antes do exercício onde estão os códigos da criação. Mas antes de inserir os dados na tabela é bom ver quais os campos existentes, o comando usado pode ser o DESCRIBE.

Agora vamos inserir na coluna nome “Francisco” no endereço “Rua: 24 de Maio nº 324” e na cidade “Fortaleza”. Observe que não é preciso inserir dados no campo “cliente_id”, pois ele foi definido como AUTO_INCREMENT.

INSTRUÇÕES DML – INSERT

```
mysql> USE venda;
```

```
Database changed
```

```
mysql> DESCRIBE cliente;
```

Field	Type	Null	Key	Default	Extra
cliente_id	int(8)	NO	PRI	NULL	auto_increment
nome	varchar(50)	NO		NULL	
endereco	varchar(80)	NO		NULL	
cidade	varchar(30)	NO		NULL	

```
4 rows in set (0.01 sec)
```

```
mysql> INSERT INTO cliente(nome,endereco,cidade)
```

```
    -> VALUES('Francisco','Rua: 24 de Maio nº 324','Fortaleza');
```

```
Query OK, 1 row affected (0.05 sec)
```

INSTRUÇÕES DML – INSERT

Temos também a opção de omitir as declarações dos campos. Essa sintaxe funciona somente se forem repassados valores para todas as colunas.

INSERT INTO <nome da tabela> VALUES (valor1, valor2, valor3,...);

Cada valor é inserido no campo que corresponde a sequência das colunas na tabela, se a primeira coluna, por exemplo, for o nome então o valor1 deve ser o nome, se a segunda coluna, por exemplo, for idade então o valor2 deve ser a idade e assim por diante.

OBS: Se a coluna for declarada como AUTO_INCREMENT basta você colocar o valor que corresponde a essa coluna como sendo zero(0). O valor zero(0) não influencia em nada porem se ele for esquecido vai ser gerado um erro.

INSTRUÇÕES DML – INSERT

Vamos usar como exemplo a mesma tabela que usamos no exemplo anterior do banco de dados “venda” que é a tabela “cliente”, porém não vai ser preciso mencionar os nomes dos campos basta repassar valores para todas as colunas.

Vamos inserir o cliente “Pedro” que mora na “Av. Augusto dos Anjos nº 2674” na cidade de “Fortaleza”. Observe que o primeiro valor que iremos inserir será zero(0), isso porque a primeira coluna é “cliente_id” que foi declarada como AUTO_INCREMENT.

INSTRUÇÕES DML – INSERT

```
mysql> USE venda;
```

```
Database changed
```

```
mysql> DESC cliente;
```

Field	Type	Null	Key	Default	Extra
cliente_id	int(8)	NO	PRI	NULL	auto_increment
nome	varchar(50)	NO		NULL	
endereco	varchar(80)	NO		NULL	
cidade	varchar(30)	NO		NULL	

```
4 rows in set (0.01 sec)
```

```
mysql> INSERT INTO cliente
```

```
-> VALUES(0,'Pedro','Av. Augusto dos Anjos nº 2674','Fortaleza');
```

```
Query OK, 1 row affected (0.04 sec)
```

INSTRUÇÕES DML – SELECT

A forma mais simples de se fazer um SELECT é recuperando todos os dados de uma tabela. A síntese básica é:

SELECT * FROM <nome da tabela>;

OBS: O * (asterisco) substitui os nomes de todas as colunas, e todas serão selecionadas para o resultado da consulta. A instrução FROM indica de qual tabela estamos buscando os dados.

INSTRUÇÕES DML – SELECT

Caso não fosse de nosso desejo mostrar todas as colunas no resultado da consulta, bastaria nomear as colunas que deveriam aparecer no lugar do * (asterisco) e separadas por vírgula.

SELECT <coluna1>, <coluna2> FROM <nome da tabela>;

Para que possamos ver os dados dos clientes que inserimos no exemplo passado, basta usar o comando **SELECT * FROM cliente**. Se você não tem certeza que o nome da tabela é essa, use o comando **SHOW TABLES** e veja as tabelas existentes dentro do BD.

INSTRUÇÕES DML – SELECT

```
mysql> USE venda;  
Database changed  
mysql> SHOW TABLES;
```

```
+-----+  
| Tables_in_venda |  
+-----+  
| cliente          |  
| pedido           |  
+-----+
```

```
2 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM cliente;
```

cliente_id	nome	endereco	cidade
1	Francisco	Rua: 24 de Maio nº 324	Fortaleza
2	Pedro	Av. Augusto dos Anjos nº 2674	Fortaleza

```
2 rows in set (0.00 sec)
```

INSTRUÇÕES DML – SELECT

FROM	Utilizada para especificar a tabela que se vai selecionar os registros.
WHERE	Utilizada para especificar as condições que devem reunir os registros que serão selecionados.
GROUP BY	Utilizada para separar os registros selecionados em grupos específicos.
HAVING	Utilizada para expressar a condição que deve satisfazer cada grupo.
ORDER BY	Utilizada para ordenar os registros selecionados com uma ordem específica.
DISTINCT	Utilizada para selecionar dados sem repetição.

INSTRUÇÕES DML – SELECT

Exemplo WHERE

Se quisermos ver somente o nome e o endereço do cliente, como já foi dito basta colocar os nomes das colunas depois do SELECT, porém quando temos muitos registros e precisamos de uma informação específica usamos a condição WHERE que em português significa ONDE. Sempre a cláusula WHERE vem acompanhada de alguma condição, por exemplo:

SELECT nome, endereco FROM cliente WHERE cliente_id = 2 ;

Podemos ler o comando SQL da seguinte forma:

Selecione o nome e o endereço da tabela cliente onde cliente_id for igual a 2.

INSTRUÇÕES DML – SELECT

```
mysql> USE venda;  
Database changed  
mysql> SHOW TABLES;
```

```
+-----+  
| Tables_in_venda |  
+-----+  
| cliente          |  
| pedido           |  
+-----+
```

```
2 rows in set (0.03 sec)
```

```
mysql> DESCRIBE cliente;
```

Field	Type	Null	Key	Default	Extra
cliente_id	int(8)	NO	PRI	NULL	auto_increment
nome	varchar(50)	NO		NULL	
endereco	varchar(80)	NO		NULL	
cidade	varchar(30)	NO		NULL	

```
4 rows in set (0.08 sec)
```

```
mysql> SELECT nome,endereco FROM cliente WHERE cliente_id = 2;
```

nome	endereco
Pedro	Av. Augusto dos Anjos nº 2674

```
1 row in set (0.05 sec)
```


INSTRUÇÕES DML – SELECT

Exemplo ORDER BY

Até o momento vimos como obter dados de uma tabela utilizando os comandos SELECT e WHERE. Porém, frequentemente precisamos listar os dados por uma ordem em particular. Pode ser por ordem ascendente ou descendente. Para isso podemos utilizar a cláusula ORDER BY para ordenar os dados. A sintaxe básica da cláusula ORDER BY é a seguinte:

SELECT < coluna > FROM < tabela > ORDER BY < coluna >;

OBS: Por padrão o ORDER BY vem como ASC significa que os resultados serão apresentados por ordem ascendente, ou seja, do menor para o maior. Mais também pode ser DESC significa que os resultados serão apresentados por ordem descendente, para isso acontecer você precisa declara-lo.

INSTRUÇÕES DML – SELECT

Exemplo ORDER BY

Como exemplo irei usar uma tabela que criei anteriormente chamada produto:

```
mysql> select * from produto;
```

id	nome	preco
1	Arroz	6.50
2	Farinha	1.80
3	Sal	0.70
4	Açúcar	5.40
5	Biscoito	1.20
6	Leite	2.35
7	Vinagre	1.95
8	Café	4.10

8 rows in set (0.00 sec)

INSTRUÇÕES DML – SELECT

Exemplo ORDER BY

Perceba que a tabela possui 8 produtos cadastrados, suponhamos que eu preciso ver os preços de forma ascendente, então iremos selecionar o nome e o preço ordenado pelo próprio preço. veja como ficou:

```
mysql> SELECT nome,preco FROM produto ORDER BY preco ASC;
```

nome	preco
Sal	0.70
Biscoito	1.20
Farinha	1.80
Vinagre	1.95
Leite	2.35
Cafe	4.10
Acucar	5.40
Arroz	6.50

8 rows in set (0.00 sec)

INSTRUÇÕES DML – SELECT

Exemplo ORDER BY

Também posso em vez de colocar o nome da coluna que eu quero ordenar basta colocar o numero dela na sequencia selecionada, por exemplo, se eu selecionei nome e preço então nome é 1 e preço é 2 e assim por diante:

```
mysql> SELECT nome,preco FROM produto ORDER BY 2 DESC;
```

nome	preco
Arroz	6.50
Acucar	5.40
Cafe	4.10
Leite	2.35
Vinagre	1.95
Farinha	1.80
Biscoito	1.20
Sal	0.70

8 rows in set (0.00 sec)

Perceba que agora usei o DESC na coluna 2 que é preço, então ficou de forma ascendente, ou seja, do maior para o menor, podemos usar a cláusula ORDER BY de várias maneiras em situações diferentes, basta saber como usa-lo.